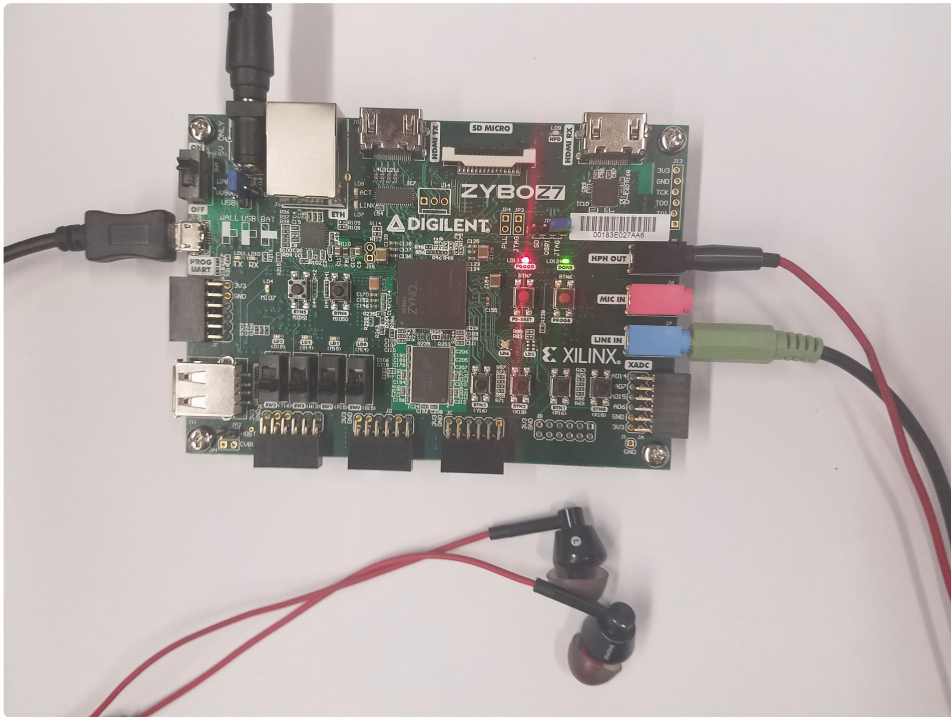


Zybo Z7 DMA Audio Demo



([https://digilent.com/reference/_detail/learn/programmable-](https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/zybo-z7-dma-audio-demo/zybo-z7-dma-action.jpg?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

[logic/tutorials/zybo-z7-dma-audio-demo/zybo-z7-dma-action.jpg?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio\)](https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/zybo-z7-dma-audio-demo/zybo-z7-dma-action.jpg?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Description

This demo contains Vivado IP Integrator and Vitis projects that control the Zybo Z7's audio codec in order to record and play audio

The audio demo records a 5-second sample from the microphone (J6) or line in (J7) port and plays it back on the headphone out (J5) port. Recording and playback are started by push buttons. Messages providing information on the status of the application are printed to a connected serial terminal.

Inventory

- Zybo Z7 with a MicroUSB Programming Cable
 - Vivado and Vitis installations compatible with the latest release of this demo (2022.1)
 - See *Installing Vivado, Vitis, and Digilent Board Files* (<https://digilent.com/reference/programmable-logic/guides/installing-vivado-and-vitis>) for installation instructions.
 - Serial Terminal application to receive messages printed by the demo
 - See *Installing and Using a Terminal Emulator* (<https://digilent.com/reference/programmable-logic/guides/serial-terminals/start>) for more information.
 - Headphones or Speakers
 - Audio Input Device - such as a microphone or a computer with 3.5mm audio jack and an audio cable
-

Download and Usage Instructions

First and foremost, releases - consisting of a set of files for download - are only compatible with a specific version of the Xilinx tools, as specified in the name of the release (referred to as a *release tag*). In addition, releases are only compatible with the specified variant of the board. For example, a release tagged “20/DMA/2020.1” for the Zybo Z7 is only to be used with the -20 variant of the board and Xilinx tools (Vivado and Vitis) version 2020.1.

The latest release version for this demo is highlighted in green.

Note: Releases for FPGA demos from before 2020.1 used a different git structure, and used a different release tag naming scheme.

Board Variant	Release Tag	Release Downloads	Setup Instructions
Zybo Z7-10	10/DMA/2022.1-1	Zybo-Z7-10-DMA-hw.xpr.zip (https://digilent.com/reference/lib/exe/fetch.php?tok=908c6f&media=https%3A%2F%2Fgithub.com%2FDigilent%2FZybo-Z7%2FReleases%2Fdownload%2F10%2FDMA%2F2022.1-1%2FZybo-Z7-10-DMA-hw.xpr.zip) Zybo-Z7-10-DMA-sw.ide.zip (https://digilent.com/reference/lib/exe/fetch.php?tok=1d51e4&media=https%3A%2F%2Fgithub.com%2FDigilent%2FZybo-Z7%2FReleases%2Fdownload%2F10%2FDMA%2F2022.1-1%2FZybo-Z7-10-DMA-sw.ide.zip)	See <i>Using the Latest Release</i> , below
Zybo Z7-20	20/DMA/2022.1-1	Zybo-Z7-20-DMA-hw.xpr.zip (https://digilent.com/reference/lib/exe/fetch.php?tok=1afc06&media=https%3A%2F%2Fgithub.com%2FDigilent%2FZybo-Z7%2FReleases%2Fdownload%2F20%2FDMA%2F2022.1-1%2FZybo-Z7-20-DMA-hw.xpr.zip) Zybo-Z7-20-DMA-sw.ide.zip (https://digilent.com/reference/lib/exe/fetch.php?tok=8d348c&media=https%3A%2F%2Fgithub.com%2FDigilent%2FZybo-Z7%2FReleases%2Fdownload%2F20%2FDMA%2F2022.1-1%2FZybo-Z7-20-DMA-sw.ide.zip)	See <i>Using the Latest Release</i> , below
Zybo Z7-10	10/DMA/2021.1-1	Zybo-Z7-10-DMA-hw.xpr.zip (https://digilent.com/reference/lib/exe/fetch.php?tok=a2aede&media=https%3A%2F%2Fgithub.com%2FDigilent%2FZybo-Z7%2FReleases%2Fdownload%2F10%2FDMA%2F2021.1-1%2FZybo-Z7-10-DMA-hw.xpr.zip) Zybo-Z7-10-DMA-sw.ide.zip (https://digilent.com/reference/lib/exe/fetch.php?tok=559476&media=https%3A%2F%2Fgithub.com%2FDigilent%2FZybo-Z7%2FReleases%2Fdownload%2F10%2FDMA%2F2021.1-1%2FZybo-Z7-10-DMA-sw.ide.zip)	See <i>Using the Latest Release</i> , below
Zybo Z7-20	20/DMA/2021.1-1	Zybo-Z7-20-DMA-hw.xpr.zip (https://digilent.com/reference/lib/exe/fetch.php?tok=8253e0&media=https%3A%2F%2Fgithub.com%2FDigilent%2FZybo-Z7%2FReleases%2Fdownload%2F20%2FDMA%2F2021.1-1%2FZybo-Z7-20-DMA-hw.xpr.zip) Zybo-Z7-20-DMA-sw.ide.zip (https://digilent.com/reference/lib/exe/fetch.php?tok=4efe74&media=https%3A%2F%2Fgithub.com%2FDigilent%2FZybo-Z7%2FReleases%2Fdownload%2F20%2FDMA%2F2021.1-1%2FZybo-Z7-20-DMA-sw.ide.zip)	See <i>Using the Latest Release</i> , below

Board Variant	Release Tag	Release Downloads	Setup Instructions
Zybo Z7-10	10/DMA/2020.1-1	Zybo-Z7-10-DMA-hw.xpr.zip Zybo-Z7-10-DMA-sw.ide.zip	See <i>Using the Latest Release</i> , below
Zybo Z7-20	20/DMA/2020.1-1	Zybo-Z7-20-DMA-hw.xpr.zip Zybo-Z7-20-DMA-sw.ide.zip	See <i>Using the Latest Release</i> , below
Zybo Z7-10	v2018.2-1	📄 Release ZIP downloads (https://github.com/Digilent/Zybo-Z7-10-DMA/releases/tag/v2018.2-1)	📄 v2018.2-1 README (https://github.com/Digilent/Zybo-Z7-10-DMA/tree/v2018.2-1)
Zybo Z7-20	v2018.2-1	📄 Release ZIP downloads (https://github.com/Digilent/Zybo-Z7-20-DMA/releases/tag/v2018.2-1)	📄 v2018.2-1 README (https://github.com/Digilent/Zybo-Z7-20-DMA/tree/v2018.2-1)

Note for Advanced Users: *GitHub sources for this demo can be found in the 📄 10/DMA/master (<https://github.com/digilent/zybo-z7/tree/10/DMA/master>) and 📄 20/DMA/master (<https://github.com/digilent/zybo-z7/tree/20/DMA/master>) branches of the Zybo-Z7 repository. Further documentation on the structure of this repository can be found on this wiki's [Digilent FPGA Demo Git Repositories](https://digilent.com/reference/programmable-logic/documents/git) (<https://digilent.com/reference/programmable-logic/documents/git>) page.*

Instructions on the use of the latest release can be found in this dropdown:

Using the Latest Release

Note: *This workflow is common across many Digilent FPGA demos. Screenshots may not match the demo you are working with.*

Important: *These steps are only to be used with releases for Xilinx tools versions 2020.1 and newer. Older releases may require other flows, as noted in the table of releases.*

First, download the '*.xpr.zip' and '*.ide.zip' files from the demo release, linked above. The XPR archive contains the Vivado project used to build the hardware platform for this demo. The project can be opened, modified, and used to update the hardware platform later if so desired, but this is optional. The IDE archive contains a set of projects to be imported into a Vitis workspace.

Note: *Unlike with Vivado XPR archives, do NOT extract the Vitis project archive ('*.ide.xip'). Vitis imports sources from the archive file directly.*

Import Vitis Projects from a Release

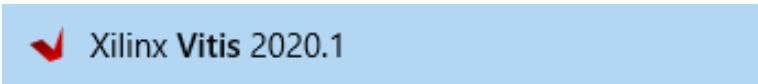
Select the dropdown corresponding to your operating system, below.

Windows

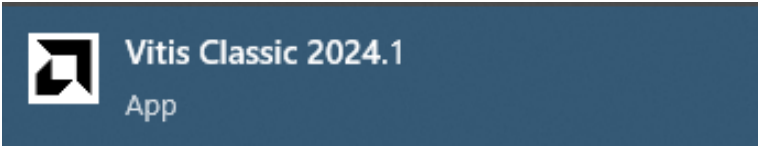
Open Vitis through the start menu or desktop shortcut created during the installation process.

Important! As of time of writing, if you're using a release for Vivado and Vitis 2024.1 or newer, Digilent only supports Vitis Classic Mode. This is an alternate user interface more similar to previous versions of the tools, which will be deprecated in upcoming versions. Digilent intends to extend support to the main Vitis user interface in the near future. To launch Classic Mode in Windows, use the dedicated Vitis Classic launcher:

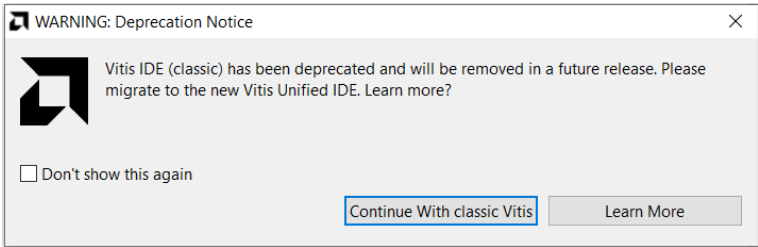
If using Vitis Classic, a deprecation notice will appear. Select **Continue With classic Vitis**.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/launch-vitis/windows.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)



(https://digilent.com/reference/_detail/programmable-logic/guides/vitis-classic.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)



(https://digilent.com/reference/_detail/programmable-logic/guides/classic-deprecation-notice.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Linux

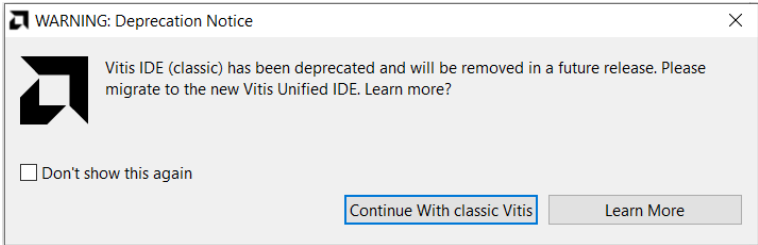
Open a terminal and run the following commands. The install path is /opt/Xilinx by default.

```
source <install_path>/Vitis/2020.1/settings64.sh
vitis
```

Important! As of time of writing, if you're using a release for Vivado and Vitis 2024.1 or newer, Digilent only supports Vitis Classic Mode. This is an alternate user interface more similar to previous versions of the tools, which will be deprecated in upcoming versions. Digilent intends to extend support to the main Vitis user interface in the near future. To launch Classic Mode in Linux, add the `--classic` flag to the `vitis` command:

```
vitis --classic
```

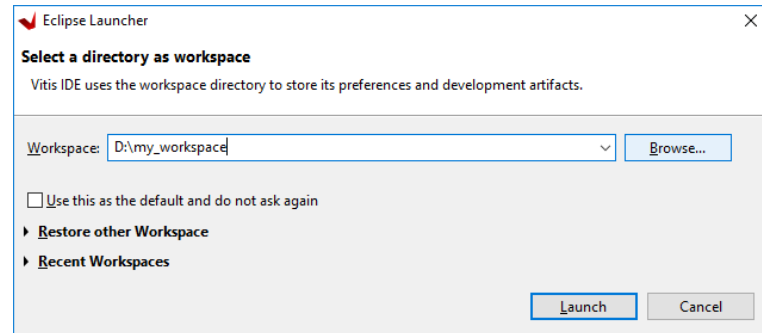
If using Vitis Classic, a deprecation notice will appear. Select **Continue With classic Vitis**.



(https://digilent.com/reference/_detail/programmable-logic/guides/classic-deprecation-notice.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

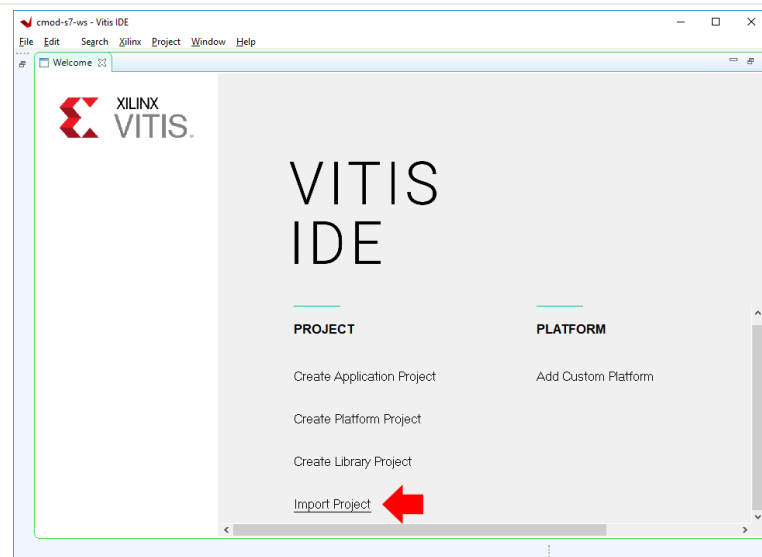
Note: Regardless of OS (), if Vivado is open, Vitis can also be launched through the Tools → Launch Vitis toolbar option.

Upon launching Vitis, a dialog will appear where a workspace must be chosen. The workspace is the directory where all of the projects and files for the application being developed will live. If a folder that does not currently exist is chosen, it will be created. Choose a workspace and click **Launch** to finish launching Vitis.



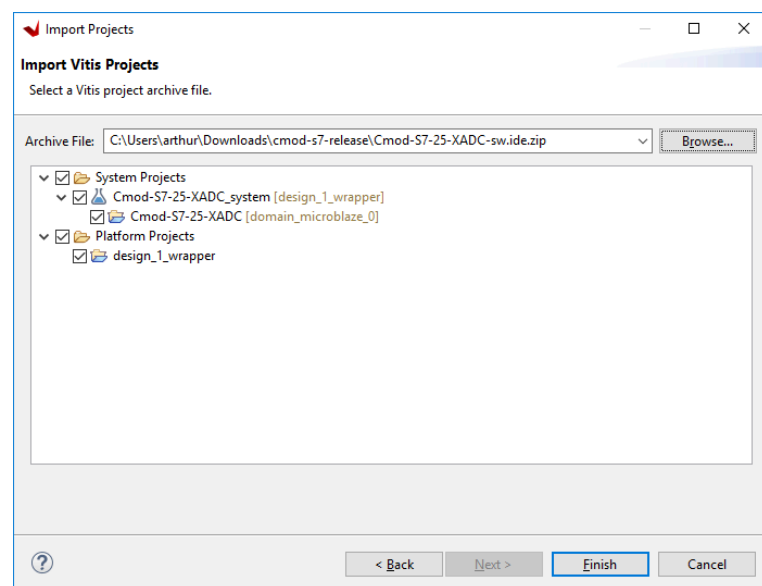
(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/launch-vitis/open-vitis-2.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

With Vitis open, click the **Import Project** button to import projects from a *Vitis project exported zip file*, then navigate to and select the IDE zip file you downloaded.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/using-releases/import-project.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

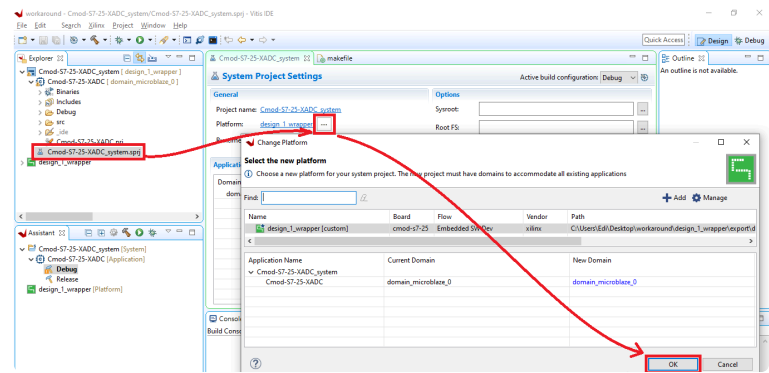
Make sure each project in the archive is checked, then click **Finish** to import them into your workspace.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/using-releases/select-all.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

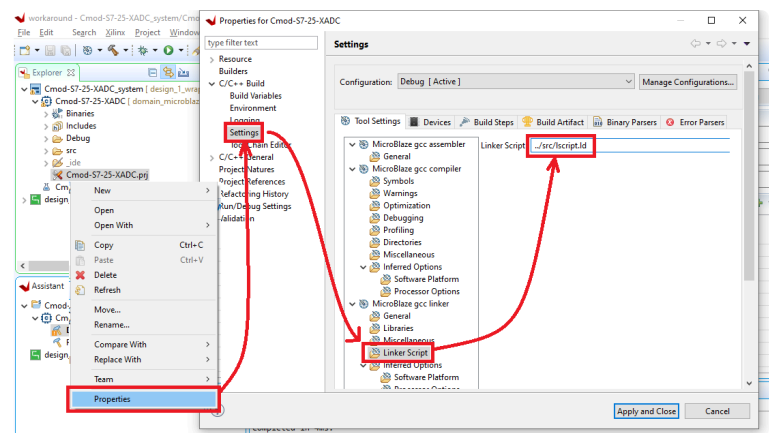
Due to a bug in some versions of Xilinx's Vitis IDE, two more steps are needed for the project to import correctly. If your projects build without errors, ignore this section.

After importing the project, open the **.sprj** file. In the System Project Settings, select a platform by clicking on the ... icon, as shown in the image. Select the platform presented and click Ok.



(https://digilent.com/reference/_detail/reference/programmable-logic/guides/workaround1.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Right click on the **.prj** file and select **Properties**. In the opened window, go into **C/C++ Build** and select **Settings**, as shown in the image. After opening the Settings tab, select **Linker Script** and change the existing path to `../src/lscript.ld`. Click Apply and Close.



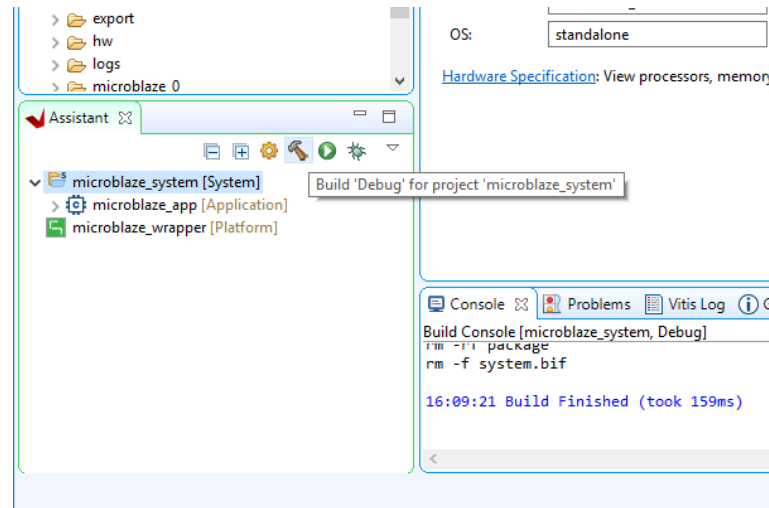
(https://digilent.com/reference/_detail/reference/programmable-logic/guides/workaround2.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Build a Vitis Application

Once an application project has been set up and includes all necessary sources, it should be built. To build the project and all of its dependencies, select the **[System]** project in the Assistant pane, and either click the **Build** button (🔧), or press Ctrl-B on your keyboard.

Note: There are three types of build targets in the Assistant pane, Platforms, Systems, and Applications. Building the application will not trigger any other applications in the system to be built, but will build the wrapper as a dependency. Building the platform will only build the platform, as it has no dependencies. Building the system causes each application in the system, as well as the platform, to be built.

This process may take several minutes to complete. When done, the Console tab at the bottom of the window will display a “Build Finished” message.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/build-software/build-system.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Set up the Zybo Z7

Plug the Zybo Z7 into the computer via the microUSB programming cable.

Plug your microphone or aux input into the appropriate jack on your Zybo (MIC IN or LINE IN, respectively). Plug your speakers/headphones into the headphone jack (HPH OUT).



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/zybo-z7-dma-audio-demo/zybo-z7-dma-action.jpg?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

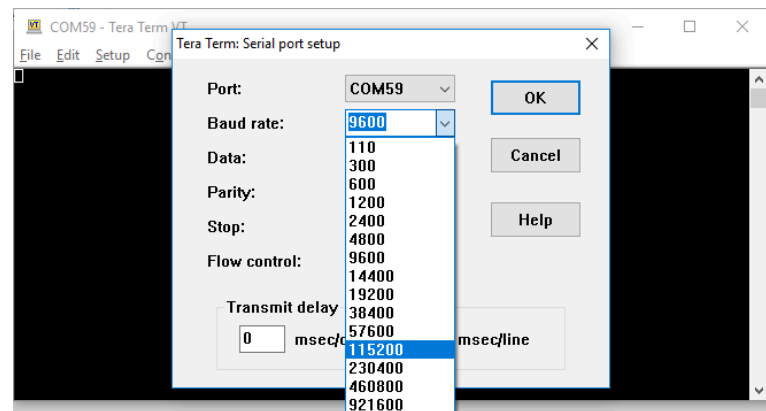
Launch a Vitis Application

Make sure your board is set to boot from JTAG before it's powered on. JTAG programming can override other boot modes on some devices, but it's easier to tell when a project is programmed into the board if there isn't already one in there.

First, many applications require that a serial console is connected to the board, so that standard output (from print statements) can be viewed. For this purpose, a serial terminal should be used. Use a serial terminal application to connect to the board's serial port. Unless otherwise stated, Zynq designs use a baud rate of 115200 and Microblaze designs with an AXI UART Lite IP use a baud rate of 9600. Flow control should be set to NONE.

Note: While Vitis has a built in serial terminal included in its Debug view, it sends characters to a board on a line-by-line basis. Some software examples require the use of character-by-character reception of data.  Tera Term (<https://ttssh2.osdn.jp/index.html.en>) or  PuTTY

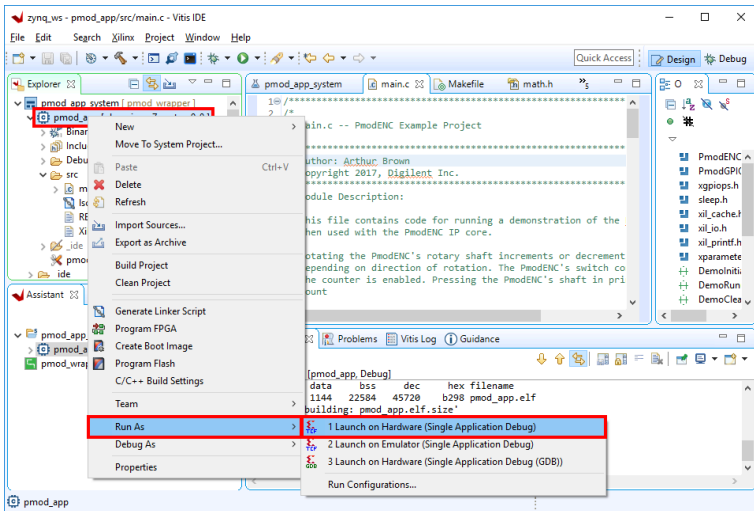
(<https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>) are recommended if you are not sure what will work.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/launch-vitis-application/set-baud.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

In the *Explorer* pane at the left side of the screen, right click on the application or system project that is to be run, and select *Run as* → *1 Launch on Hardware (Single Application Debug)*. The FPGA will be programmed with the bitstream, the ELF file created by the software build is loaded into system memory, and the application project will begin to run. You will need to click back over to the *Vitis Serial Terminal* from the *Console* tab.

Note: Once the project has been run at least once, you can use the green run button () in the toolbar at the top of the screen to program the board instead.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/launch-vitis-application/launch-on-hardware.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

At this point, the demo is now running on your board. Refer to the [Description](#) section of this document for more information on what it does.

Additional steps beyond here present how you can use the other archive provided in the release, containing the hardware project, to rebuild the Vivado project, and use a newly exported XSA file to update the platform in Vitis.

In order to modify and switch out the hardware platform for a baremetal demo, you should first open the Vivado project from the release. Extract the previously downloaded '*.xpr.zip' file.

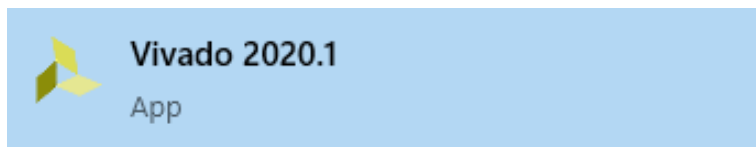
Open a Block Design Project in Vivado

Launch Vivado

Select the dropdown corresponding to your operating system, below.

Windows

Open Vivado through the start menu or desktop shortcut created during the installation process.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/launch-vivado/open-vivado.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Linux

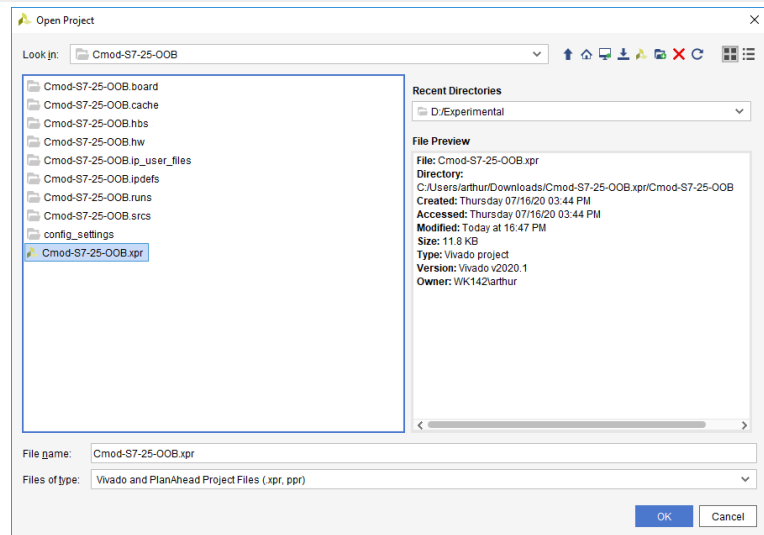
Open a terminal, and change directory (cd) to a folder where log files for your Vivado session can be placed, then run the following commands:

```
source <install_path>/Vivado/<version>/settings64
vivado
```



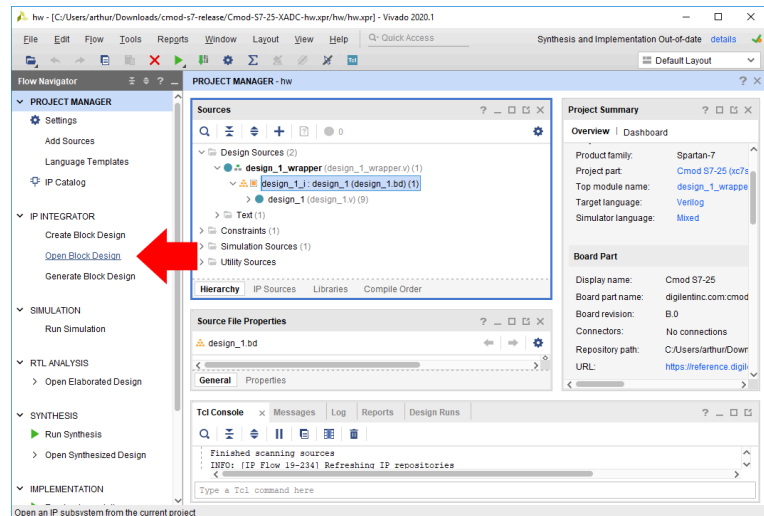

(https://digilent.com/reference/_detail/vivado/getting_started/linux_start_vivado.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

In Vivado's welcome screen, use the **Open Project** button to navigate to and open the XPR file contained in the folder the release was extracted into.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/using-releases/choose-xpr.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

The project's block diagram, which contains the design, with all of the existing components and their connections, can be opened by either double-clicking on the "*.bd" file in the sources pane (which also includes other source files, such as constraints), or by clicking the **Open Block Design** button in the *Flow Navigator* pane.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/using-releases/open-bd.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

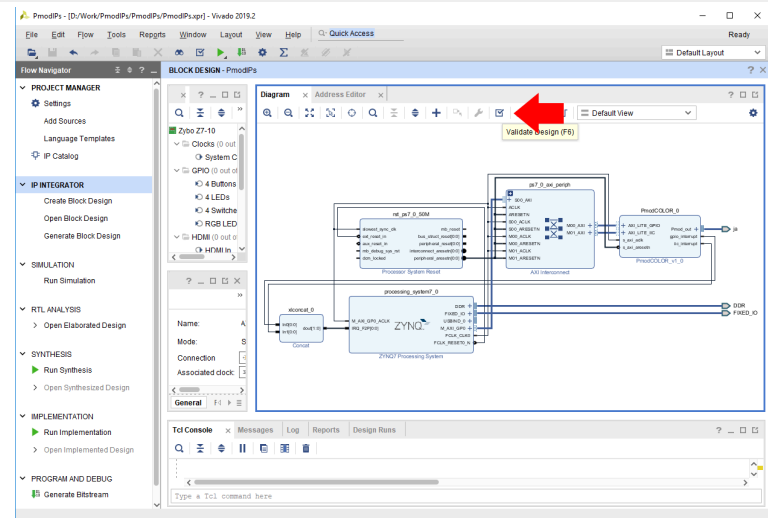
Making changes to the design is out of the scope of this particular document. More information on how to use IP Integrator to create or modify a project can be found through [Getting Started with Vivado and Vitis for Baremetal Software Projects](https://digilent.com/reference/programmable-logic/guides/getting-started-with-ipi) (<https://digilent.com/reference/programmable-logic/guides/getting-started-with-ipi>). The remainder of this document will discuss how to generate a bitstream, export a new hardware platform, and load it into Vitis.

Before the Vivado project can be built, the block design must be validated. This step runs an automatic check of the block design to see if there are any potential issues with it. Click the **Validate Design** button () (https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/validate-block-design/validate-button.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)) in the Diagram pane's toolbar (or press the F6 key).

If the design has issues, a dialog will pop up that lists them. It should be noted that most *Warnings* can be ignored, as can some *Critical Warnings*. These issues can also be viewed in the *Messages* tab of the pane at the bottom of the window.

If there are no issues, a dialog will pop up that will tell you so. Click **OK** to continue.

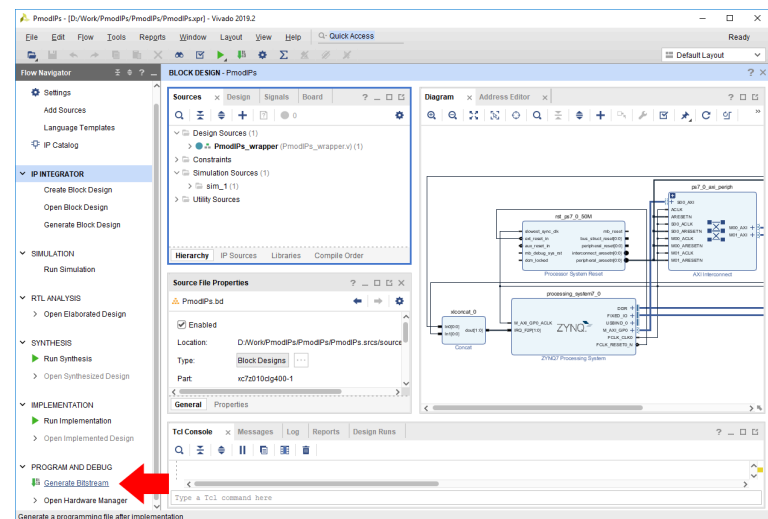
Note: Some Zynq boards may produce critical warnings at this stage relating to `PCW_UIPARAM_DDR_DQS_TO_CLK_DELAY` parameters. These warnings are ignorable and will not affect the functionality of the project. See the *Hardware Errata* section of your board's reference manual for more information.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/validate-block-design/validate-design.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Build a Vivado Project

At this point, the Vivado Project is ready to be built, by running it through Synthesis and Implementation, and finally generating a bitstream. Click the **Generate Bitstream** button in the *Program and Debug* section of the *Flow Navigator* pane at the left side of the window.



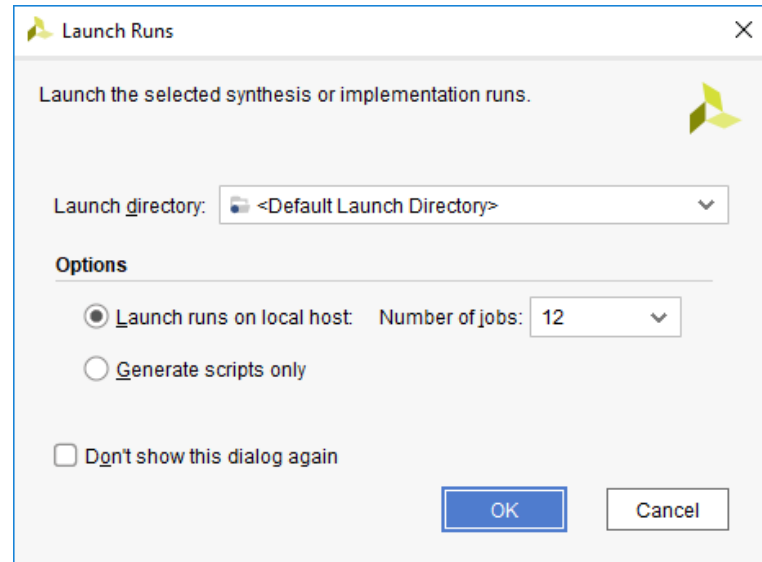
(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/generate-bitstream/generate-bitstream-1.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

A dialog will pop up with several options for how Synthesis and Implementation should be run. Most should be left as defaults. Of particular importance is the *Number of jobs* dropdown, which is used to specify how much of the resources of your computer should be dedicated to the build. A larger number of jobs will

dedicate more resources, which will allow the build to be completed faster. It is recommended to choose the highest available number.

Note: Critical warnings about how IPs included within another IP were packaged with a different board value can be safely ignored. The same is true for warnings related to negative CK-to-DQS delays seen on some Zynq boards.

Depending on the complexity of the design, the board used, and the strength of your computer, the process of building the project can take between 5 and 60 minutes.

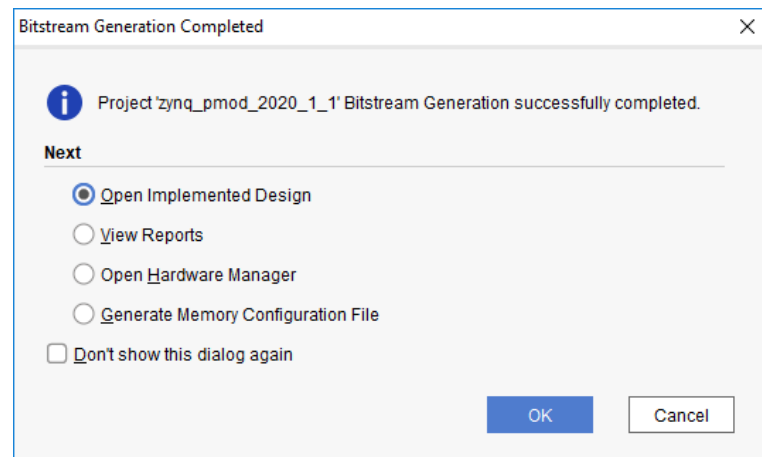


(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/generate-bitstream/generate-bitstream-2.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

When complete, a dialog will pop up that presents several options for what to do next:

- *Open Implemented Design* can be used to view the actual hardware design that has been implemented and will be placed onto the chip.
- *View Reports* can be used to view additional information about the design, including how much of the resources of the FPGA will be used by the design.
- *Open Hardware Manager* can be used to go directly to Vivado's Hardware Manager, which can be used to program a hardware design onto a board. This is typically used for designs that do not involve a software component.
- *Generate Memory Configuration File* can be used to create a file for programming an FPGA-only design into flash memory.

If none of these options are desired, click **Cancel** to continue.

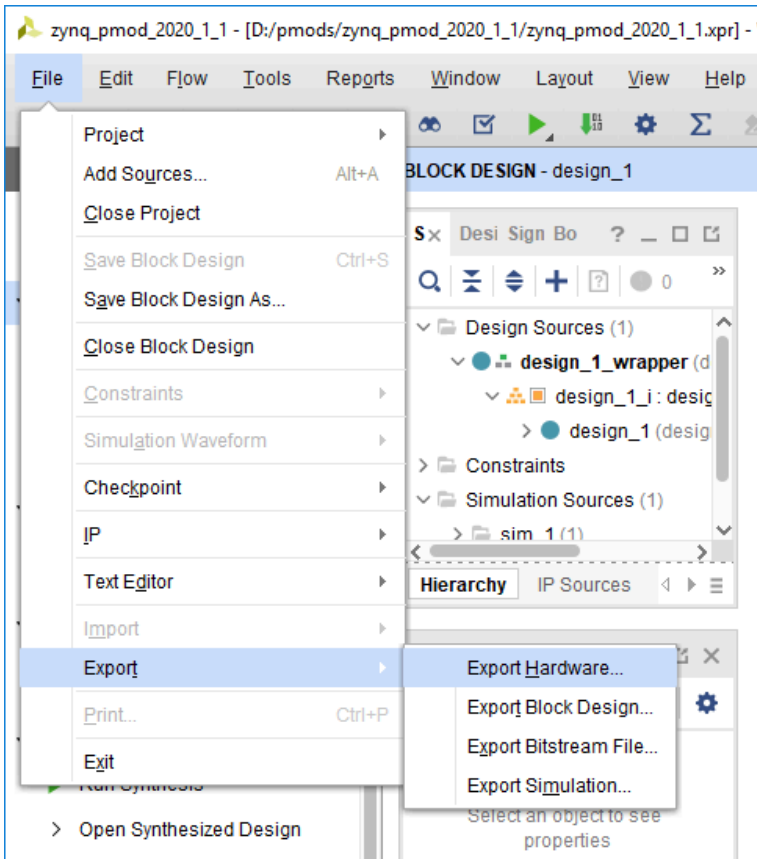


(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/generate-bitstream/generate-bitstream-3.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Export a Hardware Platform

Once the project has been built, the design must be exported from Vivado so that Vitis has access to information about the hardware that a software application is being developed for. This includes the set of IP connected to the processor, their drivers, their addresses, and more. Exporting hardware after the bitstream has been generated allows you to program your board directly from within Vitis.

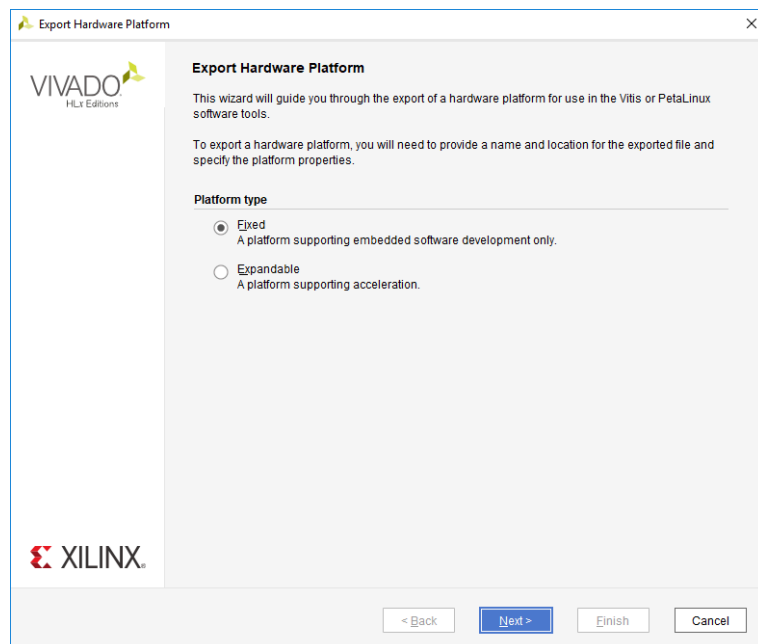
To export the hardware design, click **Export** → **Export Hardware** in the *File* menu.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/export-fixed-hardware/export-hardware.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

The wizard that pops up guides you through the options available for hardware export. The first screen allows you to select a *Fixed* or *Expandable* platform. In this case, choose a Fixed platform and click **Next** to continue.

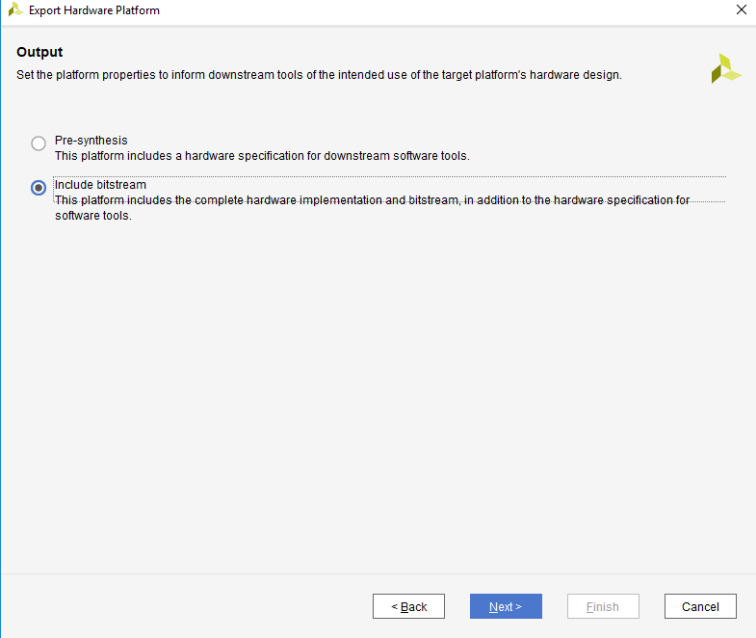
This screen is not present in Vivado 2022.1, proceed to the next



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/export-fixed-hardware/export-hardware-fixed-1.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

The *Output* screen allows you to select whether only the hardware specification (*Pre-synthesis*) should be exported, or whether the bitstream should be included. Since the bitstream has already been

generated, it should be included in the platform so that Vitis can automatically figure out where it is when programming a board. Select *Include bitstream* and click **Next** to continue.

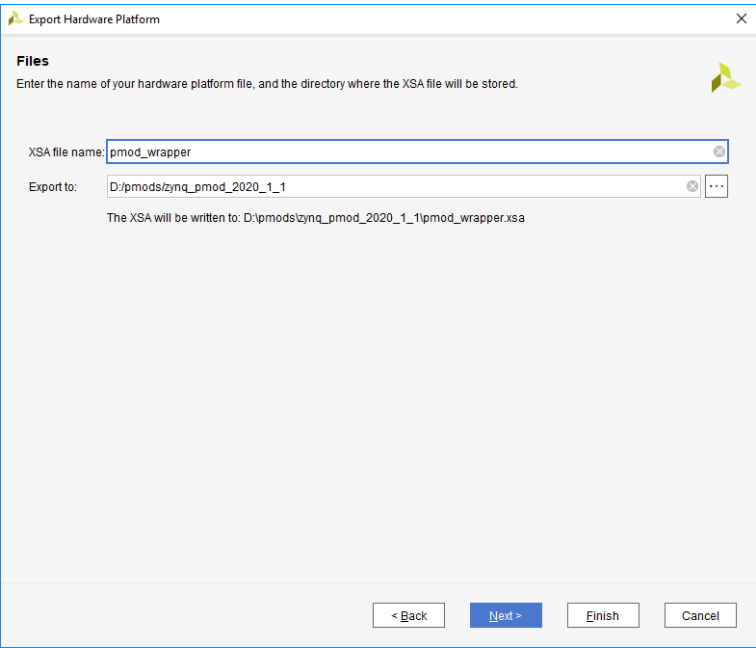


(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/export-fixed-hardware/export-hardware-fixed-2.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

The *Files* screen gives you the option to choose a name for the Xilinx Shell Architecture (XSA) file, and provide a path to a folder that the file will be placed within. Give your XSA file a name, and choose a memorable location to place it in. This file will later be imported into Vitis, so take a note of where it is placed and what it is called.

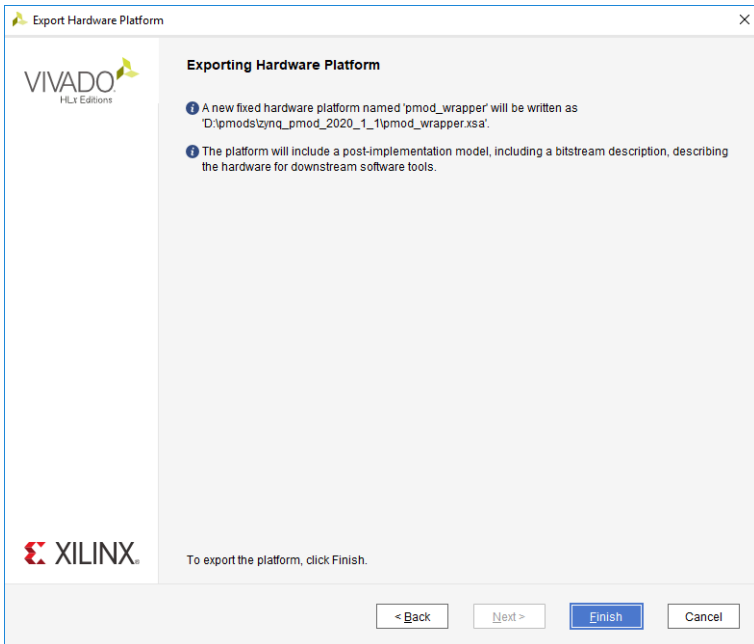
Important: *Do not use spaces in the file name or export path. Underscores or  camelCase (https://en.wikipedia.org/wiki/Camel_case) are recommended instead.*

Click **Next** to continue.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/export-fixed-hardware/export-hardware-fixed-3.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

The final screen of the wizard summarizes the options you selected. Click **Finish**.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/export-fixed-hardware/export-hardware-fixed-4.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

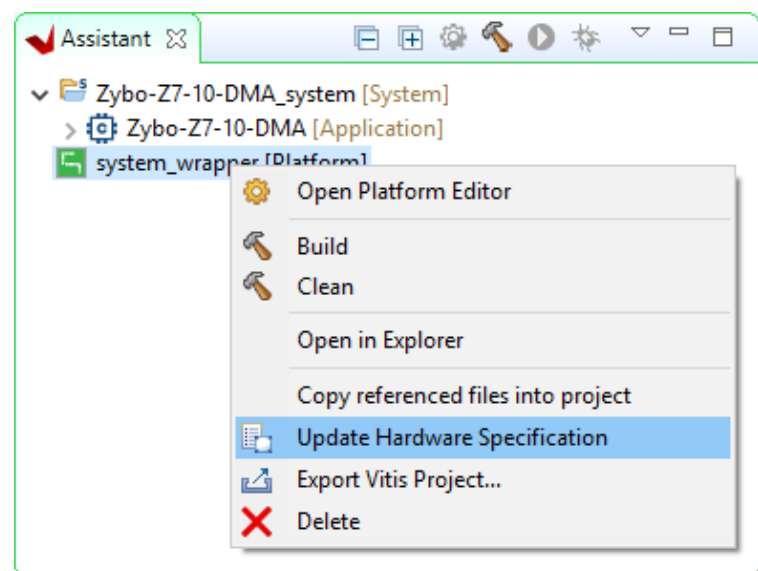
Update a Hardware Platform in Vitis

If a hardware design is changed after having created a Vitis application project, several steps must be taken in order to update the Vitis workspace with a newly exported XSA file. The XSA file contains all of the information relevant to Vitis about the hardware platform, and changing a platform project's specification based on this file will automatically load in any changes. This includes adding new drivers for new IP that have been installed and changing the files that define the addresses and other details of any installed IP that may have been renamed or had their addresses changed.

These steps assume that you have already regenerated the bitstream and reexported hardware in the same way that would be done prior to creating a new Vitis workspace.

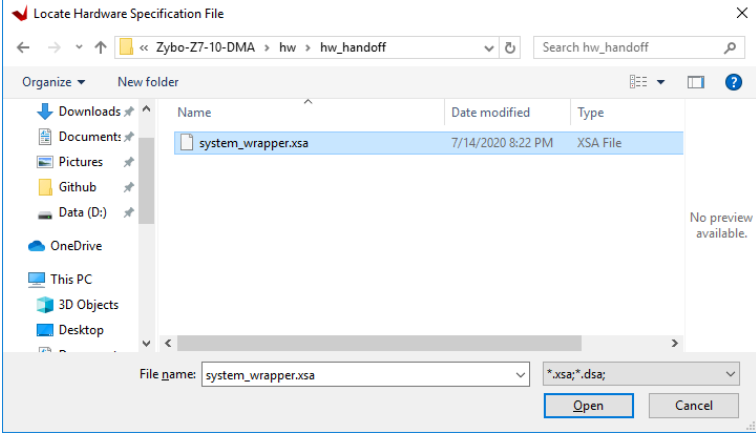
Within Vitis' Assistant pane, find the platform project that you wish to update with the new hardware. This project will typically have a name that ends with “_wrapper”, and is marked with the text “[Platform]”.

Right click on this project and select *Update Hardware Specification*.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/update-hw-spec/right-click-menu.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

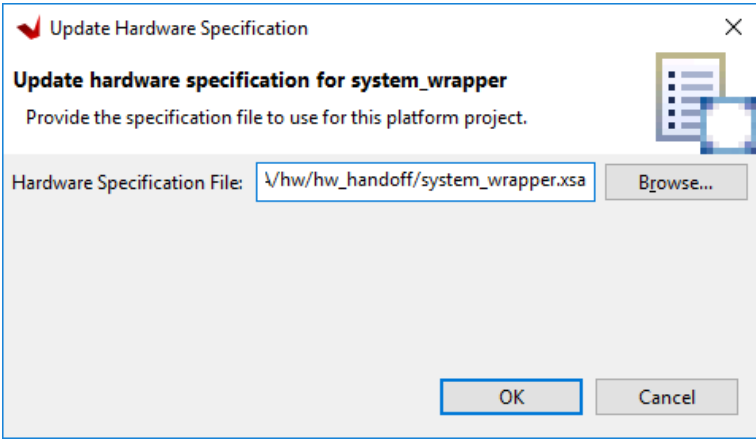
In the dialog that pops up, click **Browse**, and navigate to the location of the XSA file that you want the platform to target. Click **Open** to select this file.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/update-hw-spec/browse-to-xsa.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

Double check that the *Hardware Specification File* path matches that of the XSA file you want to use, then click **OK** to start the automatic process of updating the platform.

When complete, a dialog will pop up to state that the platform project has been updated. Click **OK** to acknowledge this.



(https://digilent.com/reference/_detail/learn/programmable-logic/tutorials/2020.1/update-hw-spec/check-path.png?id=programmable-logic%3Azybo-z7%3Ademos%3Adma-audio)

At this point, changes to the hardware specification have been loaded into the hardware platform. The bitstream will have been updated, if it was loaded into the XSA file. The set of drivers and the xparameters file will have changed to match what is in the modified design. Changes to the software application may be required before the application can be built and programmed onto the board, however, detailing what may need to be done is outside of the scope of this guide.

Controls

Audio playback and recording are controlled by push buttons as below.

Button	Function
BTN0	no effect
BTN1	record from mic in
BTN2	play on hph out
BTN3	record from line in

Recording from an Input

To record from MIC IN, press **BTN1**. To record from LINE IN, press **BTN3**. Once the recording is activated, the message “Start Recording...” will be sent over UART and the demo will record 5 seconds of audio. If any buttons are pressed during the recording, the message “Still Recording...” will be sent over UART.

Playing to an Output

To play to HPH OUT, press **BTN2**. Once the playback is activated, the message “Start Playback...” will be sent over UART and the demo will play 5 seconds of audio. If any buttons are pressed during the playback, the message “Still Playing...” will be sent over UART.

Additional Resources

All materials related to the use of the Zybo Z7 can be found on its [Resource Center \(https://digilent.com/reference/programmable-logic/zybo-z7/start\)](https://digilent.com/reference/programmable-logic/zybo-z7/start).

For a walkthrough of the process of creating a simple baremetal software project in Vivado and Vitis, see [Getting Started with Vivado and Vitis for Baremetal Software Projects \(https://digilent.com/reference/programmable-logic/guides/getting-started-with-ipi\)](https://digilent.com/reference/programmable-logic/guides/getting-started-with-ipi). Information on important parts of the GUIs, and indirect discussion of the steps required to modify, rebuild, and run this demo in hardware can also be found here.

For technical support, please visit the [🔗 FPGA \(https://forum.digilent.com/forum/4-fpga/\)](https://forum.digilent.com/forum/4-fpga/) section of the Digilent Forum.

[Company \(https://digilent.com/company/\)](https://digilent.com/company/)

- [About Us \(https://digilent.com/company/#about-digilent\)](https://digilent.com/company/#about-digilent)
- [FAQs \(https://digilent.com/company/#faqs\)](https://digilent.com/company/#faqs)
- [Distributors \(https://digilent.com/shop/distributors/\)](https://digilent.com/shop/distributors/)
- [Shipping & Returns \(https://digilent.com/shipping-returns/\)](https://digilent.com/shipping-returns/)
- [Jobs \(https://digilent.com/company/#jobs\)](https://digilent.com/company/#jobs)
- [Legal & Privacy \(https://digilent.com/legal-privacy/\)](https://digilent.com/legal-privacy/)

[News \(https://digilent.com/news/\)](https://digilent.com/news/)

- [Blog \(https://digilent.com/blog/\)](https://digilent.com/blog/)
- [Newsletter \(https://digilent.com/news/#newsletter\)](https://digilent.com/news/#newsletter)
- [Events \(https://digilent.com/news/#events\)](https://digilent.com/news/#events)

Subscribe to our newsletter

Get the latest updates on new products and upcoming sales

Contact Us

- [Technical Support Forum \(https://forum.digilent.com\)](https://forum.digilent.com)
- [Support Channels \(https://digilent.com/support/#channels\)](https://digilent.com/support/#channels)

Digilent

1300 NE Henley Ct. Suite 3
Pullman, WA 99163
United States of America

- <http://twitter.com/DigilentInc>
- <http://facebook.com/Digilent>
- <https://www.youtube.com/user/DigilentInc>
- <https://github.com/digilent>
- <https://instagram.com/digilentinc>
- <https://www.linkedin.com/company/1454013>
- <https://www.flickr.com/photos/127815101@N07>